

Software Documentation for SnapTray

Table of Contents

- [Software Documentation for SnapTray](#)
 - [Table of Contents](#)
 - [1. Introduction](#)
 - [2. System Overview](#)
 - [3. Requirements Specification](#)
 - [4. System Architecture](#)
 - [4.1 Components/Modules](#)
 - [4.2 Data Flow](#)
 - [4.3 Technologies](#)
 - [5. Design](#)
 - [5.1 Design Principles](#)
 - [5.1.1 Modularity and Separation of Concerns](#)
 - [5.1.2 Security-First Approach](#)
 - [5.1.3 Scalability and Performance](#)
 - [5.1.4 User-Centric Design](#)

- 5.1.5 Reliability and Fault Tolerance
- 5.1.6 Maintainability and Extensibility
- 5.1.7 Data Integrity and Consistency
- 5.1.8 Cross-Platform Compatibility
- 5.2 Database Design
 - 5.2.1 Az adatbázis célja, funkciója és a benne tárolt információk összefoglalása
 - 5.2.2 Adatbázis-terv és séma
 - 5.2.2.1 Entitások és kapcsolatok (ER modell összefoglaló)
 - 5.2.2.2 Relációs séma (táblázatok részletei)
 - 5.2.2.2.1 User (Felhasználók)
 - 5.2.2.2.2 Payment (Kifizetések)
 - 5.2.2.2.3 MenuItems (Menüelemek)
 - 5.2.2.2.4 Order (Rendelések)
 - 5.2.2.2.5 OrderItems (Rendelés tételek)
 - 5.2.2.2.6 Review (Értékelések)
 - 5.2.2.2.7 DailyMenu (Napi menü)
 - 5.2.2.2.8 ParentStudent (Szülő-Diák kapcsolat)
 - 5.2.2.2.9 SecurityLogs (Biztonsági naplók)
 - 5.2.2.2.10 UserLoyalty (Hűségprogram)
 - 5.2.2.3 Fizikai és logikai szerkezet

- 5.2.2.4 Használati esetek (Use Cases) és forgatókönyvek
- 5.2.2.5 Biztonság és hozzáférés
- 5.2.2.6 Karbantartás és üzemeltetés
- 5.3 Algorithms and Data Structures
 - 5.3.1 Data Structures
 - 5.3.1.1 MongoDB Collections (NoSQL Documents)
 - 5.3.1.2 Redis Data Structures
 - 5.3.1.3 Redis Lua Scripting
 - 5.3.1.4 JavaScript Objects and Arrays
 - 5.3.2 Core Algorithms
 - 5.3.2.1 Authentication and Security Algorithms
 - 5.3.2.2 Payment Processing Algorithms
 - 5.3.2.3 Loyalty Point Calculation Algorithm
 - 5.3.2.4 Search and Filtering Algorithms
 - 5.3.2.5 Caching Algorithms
 - 5.3.2.6 Rate Limiting Algorithm
 - 5.3.2.7 Data Validation Algorithms
 - 5.3.3 Performance Optimization Algorithms
 - 5.3.3.1 Database Query Optimization
 - 5.3.3.2 Pagination Algorithm

- 5.3.3.3 Batch Processing Algorithm
- 5.3.4 Security Algorithms
 - 5.3.4.1 reCAPTCHA Verification
- 5.3.5 Algorithm Selection Rationale
- 5.4 Security Design
- 6. Implementation
- 7. Testing and Validation
- 8. User Manual
- 9. Deployment and Maintenance
- 10. Conclusion and Future Work
- 11. References
- 12. Appendices

1. Introduction

A SnapTray egy webalapú menza-rendelőrendszer, amelynek célja, hogy egyszerűsítse az étkezési rendelések lebonyolítását iskolai környezetben. Fő célja, hogy a diákok, szülők és az étkeztető személyzet közötti interakció gyorsabbá és átláthatóbbá váljon az online rendelés, a valós idejű rendeléskövetés és a biztonságos fizetési lehetőségek révén.

A rendszer három fő felhasználói szerepet szolgál ki. A diákok böngészhetnek az étlapok között, rendelhetnek, kezelhetik a virtuális pénztárcájukat, és nyomon követhetik tranzakcióikat. A szülők figyelemmel kísérhetik gyermekeik rendeléseit, kezelhetik a fizetéseket és

követhetik a költségeiket. Az adminisztrátorok számára dashboard biztosít lehetőséget az étlapok kezelésére, a rendelések nyomon követésére és statisztikák elemzésére.

A SnapTray modern biztonsági megoldásokat alkalmaz, beleértve a kétlépcsős azonosítást, az email-ellenőrzést, valamint a gyakori webes támadások elleni védelmet. A fizetési lehetőségek PayPal és Google Pay integráción keresztül biztosítottak, így a tranzakciók gyorsak és biztonságosak.

Az intuitív felhasználói felület, a megbízható háttérrendszer és a hatékony adatkezelés révén a SnapTray javítja a menzai élményt minden felhasználó számára, növelve a kényelmet, az átláthatóságot és az üzemeltetési hatékonyságot.

2. System Overview

A rendszer egy webalapú rendelési és fizetési platform oktatási intézmények számára. Célja, hogy a diákok biztonságosan tudjanak rendeléseket leadni, a szülők felügyelhessék a költségeket, az adminisztrátorok pedig kezelhessék a teljes rendszert.

Az alkalmazás kliens–szerver architektúrát követ. A backend Node.js alapú Express szerver, a frontend React komponensekből épül fel szerepkör-alapú dashboardokkal. A rendszer Redis-t használ gyorsítótárazásra, rate limitingre és Lua scriptek segítségével atomi műveletekhez. MongoDB adatbázis biztosítja az adatok perzisztens tárolását.

Megvalósított fő funkciók:

Felhasználókezelés és autentikáció:

- Többszerepkörű felhasználói rendszer (diák, szülő, adminisztrátor)
- Email alapú fiókellenőrzés és kétlépcsős azonosítás (2FA)
- JWT alapú munkamenet-kezelés Redis tárolással
- Jelszó biztonság bcrypt hasheléssel és erősség validációval

Rendelés és menü rendszer:

- Dinamikus menükezelés kategóriákkal és táplálkozási információkkal
- Valós idejű készletkövetés riasztásokkal
- Napi menü funkcionalitás menüelemek társításával
- QR kód integráció menüelemekhez
- Átfedhetetlen allergén és étrendi információk

Fizetés feldolgozás:

- PayPal és Google Pay API integráció
- Pénztárca egyenleg rendszer atomi műveletekkel
- Helyszín alapú pénznem támogatás
- Biztonságos tranzakció naplózás és audit trail

3. Requirements Specification

Fő célok

- Biztonságos és ellenőrzött rendelési folyamat
- Digitális pénztárca és hűségpont rendszer
- Skálázható és nagy teljesítményű backend
- Külső fizetési szolgáltatók integrálása (PayPal, Google Pay)

Felhasználókezelés

- A rendszernek lehetővé kell tennie a felhasználók regisztrációját.
- A rendszernek támogatnia kell az email alapú fiókellenőrzést.
- A rendszernek szerepkör-alapú hozzáférést kell biztosítani (diák, szülő, admin).

Hitelesítés és biztonság

- A rendszernek JWT alapú hitelesítést kell alkalmaznia.
- A rendszernek támogatnia kell a kétlépcsős azonosítást (2FA).
- A rendszernek védenie kell a brute-force támadások ellen rate limiting segítségével.

Rendelések és fizetések

- A rendszernek lehetővé kell tennie a diákok számára rendelés leadását.
- A rendszernek ki kell számítani a rendelés végösszegét.

- A rendszernek támogatnia kell PayPal és Google Pay fizetéseket.
- A rendszernek nyilván kell tartania a tranzakciókat.

Adminisztráció

- Az adminisztrátoroknak lehetőséget kell biztosítani felhasználók kezelésére.
- A rendszernek statisztikákat és riportokat kell biztosítania.

4. System Architecture

A rendszer három fő rétegből áll:

Megjelenítési réteg (Frontend)

- React.js komponensekből épülő felhasználói felület
- Szerepkör-alapú dashboardok (diák, szülő, admin)
- Tailwind CSS responsive design
- REST API-n keresztüli kommunikáció a backenddel
- Valós idejű frissítések Socket.IO-val

Alkalmazási réteg (Backend)

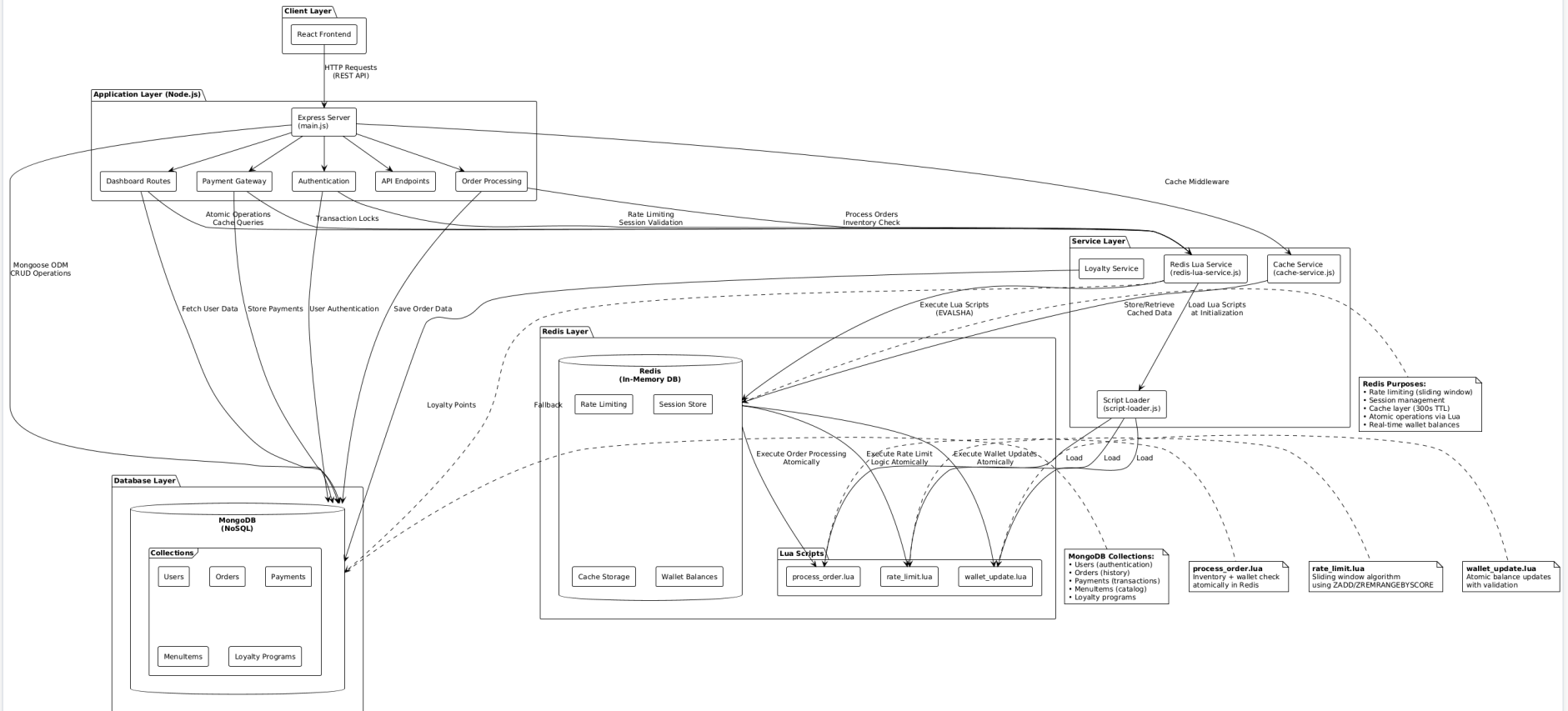
- Node.js és Express.js alapú szerver

- JWT alapú hitelesítés és jogosultságkezelés
- Üzleti logika szolgáltatásokban (order-service, payment services)
- Redis Lua scriptek atomi műveletekhez
- Rate limiting és security middleware

Adatréteg

- MongoDB perzisztens adatbázis Mongoose ODM-mel
- Redis cache munkamenetekhez és rate limitinghez
- Stratégiai indexek teljesítmény optimalizáláshoz
- Tranzakciók adat konzisztencia biztosításához

MERN Stack with Redis + Lua Architecture Repository: bivorkz/iskolaprojekt



Data Flow:

- Client → Express Server
- Redis cache check first
- Rate limiting via Lua
- Atomic operations in Redis
- Persistent data in MongoDB

Stack:
MongoDB + Express + React + Node.js + Redis + Lua

4.1 Components/Modules

The SnapTray system is organized into several key subsystems that work together to provide a cohesive cafeteria management solution:

- **Frontend/UI Layer:** Built with React.js, this layer handles user interactions, displays the interface, and manages client-side state. It includes components for authentication, menu browsing, ordering, and dashboard management.
- **API Layer:** Implemented using Node.js and Express.js, this layer provides RESTful endpoints for data access and business logic execution. It handles requests from the frontend, processes them, and returns appropriate responses.
- **Data Layer:** Comprises MongoDB for persistent data storage and Redis for caching and session management. This layer ensures data integrity, efficient retrieval, and temporary storage for performance optimization.
- **Authentication and Security Module:** Integrated across all layers, this module manages user authentication, authorization, rate limiting, and security logging.
- **Payment Processing Module:** Handles integration with external payment providers like PayPal and Google Pay, ensuring secure and reliable transaction processing.
- **Loyalty System Module:** Manages user points, tiers, and discounts, calculating rewards based on purchase history and applying appropriate benefits.

4.2 Data Flow

Data flows through the SnapTray system in a structured manner to ensure security, performance, and consistency:

1. **User Interaction:** Users interact with the React frontend, triggering API calls via HTTP requests.
2. **Request Processing:** The Express server receives requests, validates them through middleware (authentication, rate limiting, input sanitization), and routes them to appropriate handlers.

3. **Business Logic Execution:** Handlers execute business logic, often involving database queries to MongoDB or cache checks in Redis.
4. **Data Retrieval/Storage:** For read operations, data is fetched from Redis cache if available, otherwise from MongoDB. Write operations update both database and invalidate relevant cache entries.
5. **Response Generation:** Processed data is formatted and sent back to the frontend as JSON responses.
6. **Real-time Updates:** For certain operations (like wallet balance changes), Redis pub/sub is used to notify connected clients of updates.
7. **Logging and Monitoring:** All significant actions are logged to SecurityLogs collection for audit and monitoring purposes.

This flow ensures that sensitive operations are atomic, cached data remains consistent, and the system can handle concurrent users efficiently.

4.3 Technologies

The technology stack for SnapTray was chosen to balance development speed, performance, scalability, and maintainability:

- **MERN Stack (MongoDB, Express.js, React.js, Node.js):** Selected for its JavaScript full-stack consistency, reducing context switching and enabling shared code between frontend and backend. MongoDB's document-based structure aligns well with the flexible data requirements of a cafeteria system.
- **Redis:** Chosen for its high-performance in-memory data structure store, ideal for caching frequently accessed data (user sessions, menu items, statistics) and implementing features like rate limiting and real-time notifications. Its atomic operations ensure data consistency in high-concurrency scenarios.
- **Lua Scripting in Redis:** Utilized for executing complex, atomic operations on Redis data structures. Chosen because it allows multiple Redis commands to run as a single, uninterruptible transaction, ensuring data consistency, preventing race conditions, and improving performance by reducing network round trips between the application and Redis server.
- **Tailwind CSS:** Selected for rapid UI development with utility-first approach, ensuring responsive design and consistent styling across devices.

- **JWT for Authentication:** Provides stateless authentication, reducing server-side session storage needs and improving scalability.
- **bcrypt for Password Hashing:** Industry-standard for secure password storage, protecting against rainbow table and brute-force attacks.
- **reCAPTCHA:** Integrated to prevent automated abuse while maintaining user experience.
- **PayPal/Google Pay APIs:** Chosen for their robust security, global acceptance, and ease of integration for payment processing.
- **Socket.IO:** Implemented for real-time bidirectional communication between clients and server, enabling live chat and order status updates.
- **Express Validator:** Used for comprehensive server-side input validation and sanitization.
- **Helmet:** Security middleware for setting various HTTP headers to protect against common web vulnerabilities.
- **IPlocate.io:** Integrated for IP geolocation services to enhance security logging and fraud detection.
- **Nodemailer:** Used for email verification and notifications.
- **Artillery:** Implemented for load testing and performance validation.

The architecture follows a monolithic approach rather than microservices due to the project's scope and team size, allowing for simpler deployment, debugging, and data consistency. Horizontal scaling is achieved through MongoDB sharding and Redis clustering when needed.

5. Design

5.1 Design Principles

The SnapTray system follows several key design principles to ensure a robust, scalable, and user-friendly school cafeteria management solution. These principles guide the architecture, implementation, and maintenance of the system.

5.1.1 Modularity and Separation of Concerns

- **Component-based Architecture:** The system is built using React components for the frontend, ensuring reusable and maintainable UI elements.
- **Service Layer Abstraction:** Business logic is separated into dedicated service modules (order-service.js, paypal-service.js, etc.) to maintain clean separation between data access, business rules, and presentation layers.
- **Middleware Organization:** Security, authentication, and validation logic are implemented as Express middleware, allowing for clean request processing pipelines.

5.1.2 Security-First Approach

- **Defense in Depth:** Multiple layers of security including input validation, authentication, authorization, rate limiting, and data sanitization.
- **Principle of Least Privilege:** Users have access only to the minimum resources necessary for their role (student, parent, teacher, admin).
- **Secure by Default:** All endpoints require authentication by default, with explicit permissions granted for specific operations.
- **Data Protection:** Sensitive data (passwords, payment information) is encrypted and hashed appropriately.

5.1.3 Scalability and Performance

- **Horizontal Scaling:** The system uses MongoDB and Redis, which support horizontal scaling through sharding and clustering.
- **Caching Strategy:** Redis is employed for session management and data caching to reduce database load and improve response times.
- **Asynchronous Processing:** Non-blocking I/O operations and asynchronous programming patterns ensure the system can handle concurrent users efficiently.
- **Resource Optimization:** Database queries are optimized with proper indexing, and large datasets are paginated.

5.1.4 User-Centric Design

- **Intuitive User Interface:** Clean, responsive design using Tailwind CSS that works across devices (desktop, tablet, mobile).
- **Progressive Enhancement:** Core functionality works without JavaScript, with enhanced features for modern browsers.
- **Feedback Systems:** Clear error messages, loading states, and success confirmations to guide users through processes.

5.1.5 Reliability and Fault Tolerance

- **Error Handling:** Comprehensive error handling with graceful degradation - the system continues to function even when individual components fail.
- **Transaction Management:** Database operations use transactions where appropriate to maintain data consistency.
- **Logging and Monitoring:** Extensive logging for debugging and monitoring system health, with different log levels for different environments.
- **Backup and Recovery:** Regular database backups and recovery procedures to prevent data loss.

5.1.6 Maintainability and Extensibility

- **Clean Code Principles:** Following established coding standards, meaningful variable names, and comprehensive documentation.
- **Version Control:** Git-based development with feature branches and proper commit messages.
- **API Design:** RESTful API design with consistent naming conventions and response formats.
- **Configuration Management:** Environment-based configuration to support different deployment environments (development, staging, production).

5.1.7 Data Integrity and Consistency

- **Validation Layers:** Input validation at multiple levels (client-side, server-side, database-level).

- **Referential Integrity:** Proper use of MongoDB references and population to maintain relationships between entities.
- **Atomic Operations:** Critical operations (like payment processing and loyalty point updates) use atomic database operations.
- **Audit Trail:** Security logs track all important actions for compliance and debugging.

5.1.8 Cross-Platform Compatibility

- **Browser Support:** Compatible with modern browsers (Chrome, Firefox, Safari, Edge) with graceful degradation for older browsers.
- **Mobile Responsiveness:** Responsive design ensures usability on various screen sizes.
- **API Flexibility:** RESTful APIs that can be consumed by different clients (web, mobile apps, third-party integrations).

5.2 Database Design

5.2.1 Az adatbázis célja, funkciója és a benne tárolt információk összefoglalása

Ez az adatbázis egy iskolai büfék rendszer (MERN stack projekt) részét képezi, amely lehetővé teszi a felhasználók (diákok, szülők, tanárok) számára az étkezés megrendelését, kifizetését és értékelését. A rendszer támogatja a felhasználói autentikációt, a menükezelést, rendeléseket, kifizetéseket, hűségprogramokat, E2EE chatet és biztonsági naplózást. A fő cél az iskolai étkezés hatékony és biztonságos kezelése, beleértve a készletkezelést, értékeléseket és a pénzügyi tranzakciókat. Az adatbázis MongoDB-t használ Mongoose ODM-mel, amely egy NoSQL adatbázis, de sémákkal strukturált. Redis-t használ gyorsítótárazáshoz és ideiglenes adatokhoz.

Az adatbázis-modell típusa: NoSQL (MongoDB), lekérdezési nyelv: JavaScript (Mongoose queries). Redis in-memory adatbázis gyorsítótárazáshoz és session/ephemerális adatokhoz.

5.2.2 Adatbázis-terv és séma

Entitások és kapcsolatok (ER modell összefoglaló)

A rendszer fő entitásai és kapcsolataik:

- **User** (Felhasználó): Központi entitás, minden más entitáshoz kapcsolódik.
- **UserPersonalInfo**: Egy felhasználóhoz tartozó személyes adatok (alágyazott dokumentum).
- **MenuItems** (Menüelemek): Étkezési tételek.
- **Order** (Rendelés): Felhasználók rendelései.
- **OrderItems** (Rendelés tételek): Egy rendeléshez tartozó Menüelemek (Order-be ágyazva).
- **Payment** (Kifizetés): Pénzügyi tranzakciók.
- **Review** (Értékelés): Menüelemek értékelése (beágyazott MenuItems-ben).
- **DailyMenu** (Napi menü): Iskolai periódusok szerinti menük, több MenuItems-t tartalmaz (N:N kapcsolat linking table-lel).
- **ParentStudent** (Szülő-Diák kapcsolat): Szülők és diákok összekapcsolása.
- **SecurityLogs** (Biztonsági naplók): Események naplózása.
- **UserLoyalty** (Hűségprogram): Felhasználók pontjai, kedvezményei és hűség szintje.
- **DeviceSyncSession** (Eszköz szinkronizálási munkamenet): Eszközök közötti kulcs szinkronizálás (önálló, nem kapcsolódik más entitáshoz).
- **Message** (Üzenetek): E2EE chat üzenetek.
- **PreKey** (Előzetes kulcsok): ECDH előzetes kulcsok.
- **StorageBlob** (Tárolási blob): Titkosított üzenet/session történetek.

Kapcsolatok:

- User 1:N Payment, Order, SecurityLogs, UserLoyalty, Message (sender/recipient), PreKey, StorageBlob, ParentStudent (szülőként vagy diákként).
- MenuItems 1:N OrderItems (Order-be ágyazva), Review (beágyazott).
- Order 1:N OrderItems (beágyazott).
- DailyMenu N:M MenuItems (linking table: DailyMenuMenuItems).
- Message 1:1 PreKey (opcionális, X3DH-hez).
- StorageBlob 1:1 User (per blobType és partitionKey).
- DeviceSyncSession: önálló, nem kapcsolódik más entitáshoz.

Példa: DailyMenu és MenuItems kapcsolata (dbdiagram.io stílusban)

```
Table DailyMenu {
  _id objectid [pk]
  date date
  schoolPeriod varchar
  createdAt datetime
}

Table MenuItems {
  _id objectid [pk]
  name varchar
  // ...további mezők
}

Table DailyMenuMenuItems {
  dailyMenuId objectid [ref: > DailyMenu._id]
  menuItemId objectid [ref: > MenuItems._id]
  // Composite PK: [dailyMenuId, menuItemId]
}
```

DeviceSyncSession

Ez a tábla önálló, nem kapcsolódik más entitáshoz, csak eszköz-azonosítókat és titkosított adatokat tárol. Ez teljesen rendben van, mivel ephemeralis, session típusú adatokat kezel.

E2EE Chat és Üzenetkezelés az Adatbázisban

A rendszer egyik legösszetettebb és legnagyobb része a végpontok közötti titkosított (E2EE) chat funkció, amely több adatbázis-entitást is érint. Az üzenetküldés a Double Ratchet protokollra és X3DH kulcscserére épül, így minden üzenet és kulcsmozgás külön dokumentumként jelenik meg a MongoDB-ben. A főbb komponensek:

- **Message:** Minden elküldött üzenet egy dokumentum, amely tartalmazza a küldő és címzett felhasználó és eszköz azonosítóját, a titkosított üzenettartalmat, státuszt, időbélyegeket, valamint a Double Ratchet és X3DH protokollhoz szükséges metaadatokat (header, x3dhHeader, encryptionMetadata stb.).
- **PreKey:** Az előzetes kulcsok (OPK, SPK) a kulcscsere protokollhoz, minden eszközre és felhasználóra külön dokumentumként tárolva. Ezek gyorsan cserélődnek, használat után törlődnek.
- **StorageBlob:** Titkosított üzenet- és session-történetek, amelyek lehetővé teszik a kliensoldali visszaállítást vagy szinkronizációt. A szerver nem lát bele a tartalomba (zero-knowledge).
- **DeviceSyncSession:** Ideiglenes, eszközök közötti kulcsszinkronizálási session-ök, amelyek automatikusan törlődnek (TTL index).

A chat rendszer minden komponense úgy van kialakítva, hogy a szerver ne férjen hozzá a titkosított tartalomhoz, csak a szükséges metaadatokat tárolja. Az üzenetek, kulcsok és session-ök közötti kapcsolatok referenciákon (ObjectId) és eszközazonosítókra alapulnak. A Message kollekció indexei optimalizálják a keresést felhasználó, eszköz és státusz szerint. A PreKey és StorageBlob kollekciók magas churn-t kezelnek, mivel a kulcsok és session-ök gyorsan cserélődnek. Ez a felépítés biztosítja a biztonságos, skálázható és auditálható chat-funkciót az iskolai rendszerben.

5.3 Algorithms and Data Structures

The SnapTray system employs various algorithms and data structures to ensure efficient data processing, security, and performance. This section documents the key algorithms and data structures used throughout the system.

5.3.1 Data Structures

5.3.1.1 MongoDB Collections (NoSQL Documents)

- **Document-based Storage:** MongoDB collections store data as BSON documents, allowing flexible schemas and nested structures.
- **Embedded Documents:** Order items are embedded within Order documents for atomic operations and better read performance.
- **References:** ObjectId references link related documents (e.g., User to Order, MenuItem to Review).
- **Arrays:** Used for storing multiple values like allergens, menu items in daily menus, and discount lists.

5.3.1.2 Redis Data Structures

- **Strings:** Session storage, cached user data, and simple key-value pairs.
- **Hashes:** Complex cached objects like user statistics and dashboard data.
- **Sorted Sets:** Rate limiting with score-based expiration.
- **Lists:** Queue structures for background processing (if implemented).

5.3.1.3 Redis Lua Scripting

Redis Lua scripts are used for atomic operations that require multiple Redis commands to execute as a single, uninterruptible transaction. This ensures data consistency in high-concurrency scenarios.

Key Lua Scripts in the System:

Rate Limiting Script (rate_limit.lua):

```

-- Sliding window rate limiting using Redis sorted sets
local key = KEYS[1]
local window = tonumber(ARGV[1])
local max_requests = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

-- Remove old entries outside the window
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)

-- Count current requests in window
local current_count = redis.call('ZCARD', key)

-- Check if limit exceeded
if current_count >= max_requests then
    return {0, current_count} -- 0 = blocked, current count
end

-- Add current request
redis.call('ZADD', key, now, now)

-- Set expiration on the key (cleanup)
redis.call('EXPIRE', key, window)

return {1, current_count + 1} -- 1 = allowed, new count

```

- **Purpose:** Distributed sliding window rate limiting
- **Algorithm:** Uses Redis sorted sets for precise time-based windows
- **Benefits:** More accurate than fixed windows, handles burst traffic better

Wallet Update Script (wallet_update.lua):

```
-- Atomic wallet balance updates with validation
local wallet_key = KEYS[1]
local amount = tonumber(ARGV[1])

local current_balance = tonumber(redis.call('GET', wallet_key) or '0')
local new_balance = current_balance + amount

if new_balance < 0 then
    return redis.error_reply('INSUFFICIENT_FUNDS')
end

redis.call('SET', wallet_key, new_balance)

return new_balance
```

- **Purpose:** Thread-safe wallet balance modifications
- **Validation:** Prevents negative balances
- **Atomicity:** Single operation ensures consistency

Order Processing Script (process_order.lua):

```

-- Atomic order processing with inventory and wallet validation
local inventory_key = KEYS[1]
local wallet_key = KEYS[2]
local order_key = KEYS[3]
local quantity = tonumber(ARGV[1])
local price = tonumber(ARGV[2])
local user_id = ARGV[3]

-- Check inventory
local current_stock = tonumber(redis.call('GET', inventory_key) or '0')
if current_stock < quantity then
    return redis.error_reply('INSUFFICIENT_STOCK')
end

-- Check wallet balance
local current_balance = tonumber(redis.call('GET', wallet_key) or '0')
local total_cost = quantity * price
if current_balance < total_cost then
    return redis.error_reply('INSUFFICIENT_FUNDS')
end

-- Atomic operations: deduct inventory and balance, create order
redis.call('DECRBY', inventory_key, quantity)
redis.call('DECRBY', wallet_key, total_cost)
redis.call('HMSET', order_key, 'user_id', user_id, 'quantity', quantity, 'total_cost', total_cost)

return {current_stock - quantity, current_balance - total_cost, order_key}

```

- **Purpose:** Atomic order processing with inventory and payment validation
- **Benefits:** Prevents overselling and ensures payment consistency
- **Error Handling:** Comprehensive validation before execution

Lua Script Benefits:

- **Atomicity:** Multiple Redis operations execute as one atomic unit
- **Performance:** Reduces network round trips between application and Redis
- **Consistency:** Eliminates race conditions in concurrent operations
- **Reusability:** Scripts are cached on Redis server for repeated execution

5.3.1.4 JavaScript Objects and Arrays

- **Plain Objects:** Configuration objects, API responses, and in-memory data manipulation.
- **Arrays:** Cart management, menu filtering, and batch operations.
- **Maps:** Efficient key-value storage for caching and lookups.
- **Sets:** Unique value collections for filtering and deduplication.

5.3.2 Core Algorithms

5.3.2.1 Authentication and Security Algorithms

Password Hashing Algorithm:

```
// bcrypt with salt rounds for secure password storage
const hashedPassword = await bcrypt.hash(password, 12);
const isValid = await bcrypt.compare(password, hashedPassword);
```

- **Algorithm:** bcrypt with adaptive cost factor
- **Purpose:** Prevent rainbow table attacks and brute force attacks

JWT Token Generation and Verification:


```
// HS256 algorithm for token signing
const token = jwt.sign(payload, secretKey, { expiresIn: '24h' });
const decoded = jwt.verify(token, secretKey);
```

- **Algorithm:** HMAC-SHA256 (HS256)
- **Purpose:** Stateless authentication and secure data transmission
- **Security Features:** Expiration, issuer validation, audience restriction

IP Hashing for Privacy:

```
// SHA-256 hashing for GDPR compliance
const hashedIP = crypto.createHash('sha256').update(ipAddress).digest('hex');
```

- **Algorithm:** SHA-256 cryptographic hash
- **Purpose:** Anonymize IP addresses in logs while maintaining uniqueness
- **Compliance:** GDPR Article 32 data protection requirements

5.3.2.2 Payment Processing Algorithms

Currency Conversion Algorithm:

```
// Simple multiplication-based conversion
const convertCurrency = (amount, fromRate, toRate) => {
  return (amount / fromRate) * toRate;
};
```

- **Algorithm:** Linear conversion with exchange rates
- **Purpose:** Convert between different currencies for international payments

- **Precision:** Uses floating-point arithmetic with rounding to 2 decimal places

Payment Validation Algorithm:

```
// Multi-step validation with checksums
const validatePayment = (paymentData) => {
  return validateAmount(paymentData.amount) &&
    validateCurrency(paymentData.currency) &&
    validateChecksum(paymentData);
};
```

- **Algorithm:** Multi-condition validation with early termination
- **Purpose:** Prevent fraudulent transactions and data corruption
- **Error Handling:** Comprehensive validation with specific error messages

5.3.2.3 Loyalty Point Calculation Algorithm

Point Calculation Based on Purchase Amount:

```
const calculatePoints = (amount, tier, healthLevel, date) => {
  const basePoints = amount * 4; // 4 points per dollar
  const tierMultiplier = getTierMultiplier(tier);
  const healthBonus = healthLevel === 'HIGH' ? 1.5 : 1.0;
  const holidayBonus = isHoliday(date) ? 1.2 : 1.0;

  return Math.floor(basePoints * tierMultiplier * healthBonus * holidayBonus);
};
```

- **Algorithm:** Multiplicative point calculation with tier bonuses
- **Factors:** Purchase amount, user tier, health score, holiday periods

- **Rounding:** Floor function to ensure integer points

Tier Determination Algorithm:

```
const determineTier = (totalPoints) => {  
  if (totalPoints >= 20000) return 'PLATINUM';  
  if (totalPoints >= 8000) return 'GOLD';  
  if (totalPoints >= 2500) return 'SILVER';  
  if (totalPoints >= 1200) return 'BRONZE';  
  return 'NONE';  
};
```

- **Algorithm:** Threshold-based classification
- **Purpose:** Automatic tier promotion based on accumulated points
- **Triggers:** Real-time updates on point changes

5.3.2.4 Search and Filtering Algorithms

Menu Item Filtering Algorithm:

```
const filterMenuItems = (items, filters) => {  
  return items.filter(item => {  
    return (!filters.category || item.category === filters.category) &&  
      (!filters.priceRange || isInRange(item.price, filters.priceRange)) &&  
      (!filters.allergens || !hasAllergens(item, filters.allergens)) &&  
      (!filters.searchTerm || matchesSearch(item, filters.searchTerm));  
  });  
};
```

- **Algorithm:** Multi-criteria filtering with short-circuit evaluation

- **Purpose:** Efficient menu browsing with multiple filter options

Text Search Algorithm:

```
const matchesSearch = (item, term) => {  
  const searchTerm = term.toLowerCase();  
  return item.name.toLowerCase().includes(searchTerm) ||  
    item.description.toLowerCase().includes(searchTerm);  
};
```

- **Algorithm:** Case-insensitive substring matching
- **Purpose:** Basic text search across menu items
- **Limitations:** Simple implementation, could be enhanced with fuzzy matching

5.3.2.5 Caching Algorithms

Cache Key Generation:

```
const generateCacheKey = (userId, resource, params) => {  
  return `${userId}:${resource}:${JSON.stringify(params)}`;  
};
```

- **Algorithm:** Structured key generation with JSON serialization
- **Purpose:** Unique cache keys for different user-resource combinations
- **Collision Prevention:** Includes all relevant parameters

Cache Invalidation Strategy:

```
const invalidateUserCache = async (userId) => {  
  const pattern = `${userId}:*`;   
  const keys = await redisClient.keys(pattern);  
  if (keys.length > 0) {  
    await redisClient.del(keys);  
  }  
};
```

- **Algorithm:** Pattern-based bulk invalidation
- **Purpose:** Clear all user-related cache entries on data changes
- **Performance:** Uses Redis pattern matching for efficiency

5.3.2.6 Rate Limiting Algorithm

Sliding Window Algorithm (Redis Lua Implementation):

```

-- Uses Redis sorted sets for precise sliding window rate limiting
local key = KEYS[1]
local window = tonumber(ARGV[1])
local max_requests = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

-- Remove requests outside the current window
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)

-- Count requests in current window
local current_count = redis.call('ZCARD', key)

if current_count >= max_requests then
    return {0, current_count} -- Rate limit exceeded
end

-- Add current request and set expiration
redis.call('ZADD', key, now, now)
redis.call('EXPIRE', key, window)

return {1, current_count + 1} -- Request allowed

```

- **Algorithm:** Sliding window using Redis sorted sets
- **Benefits:** More accurate than fixed windows, handles burst traffic better
- **Implementation:** Distributed across multiple server instances
- **Data Structure:** Sorted set with timestamps as scores
 windowMs: 15 * 60 * 1000, // 15 minutes
 max: 100, // limit each IP to 100 requests per windowMs
 store: new RedisStore({ /* redis config */ })
 });

- **Algorithm**: Token bucket with fixed window
- **Purpose**: Prevent abuse and ensure fair resource usage
- **Storage**: Redis-backed for distributed rate limiting

5.3.2.7 Data Validation Algorithms

Input Sanitization Algorithm:

```
```javascript
const sanitizeInput = (input) => {
 return input
 .trim()
 .replace(/[<>]/g, '') // Basic XSS prevention
 .substring(0, 1000); // Length limiting
};
```

- **Algorithm**: Multi-step sanitization with regex replacement
- **Purpose**: Prevent injection attacks and normalize input data
- **Layers**: Client-side, server-side, and database-level validation

#### Email Validation Algorithm:

```
const validateEmail = (email) => {
 const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 return emailRegex.test(email) && email.length <= 254;
};
```

- **Algorithm**: Regular expression matching with length constraints
- **Purpose**: Ensure valid email format and prevent buffer overflow

## 5.3.3 Performance Optimization Algorithms

### 5.3.3.1 Database Query Optimization

#### Index Utilization:

```
// Compound indexes for common query patterns
User.collection.createIndex({ email: 1, isVerified: 1 });
Order.collection.createIndex({ userId: 1, orderDate: -1 });
```

- **Algorithm:** Strategic index creation based on query patterns
- **Purpose:** Reduce query execution time
- **Maintenance:** Automatic index updates on document changes

### 5.3.3.2 Pagination Algorithm

#### Cursor-based Pagination:

```
const getPaginatedResults = async (model, filter, page, limit) => {
 const skip = (page - 1) * limit;
 const results = await model.find(filter)
 .sort({ createdAt: -1 })
 .skip(skip)
 .limit(limit);
 const total = await model.countDocuments(filter);
 return { results, total, page, pages: Math.ceil(total / limit) };
};
```

- **Algorithm:** Skip-limit pagination with total count
- **Purpose:** Efficient handling of large datasets



### 5.3.3.3 Batch Processing Algorithm

#### Order Fulfillment Batch Processing:

```
const processOrderBatch = async (orders) => {
 const bulkOps = orders.map(order => ({
 updateOne: {
 filter: { _id: order._id },
 update: { status: 'Completed' }
 }
 }));
 return await Order.bulkWrite(bulkOps);
};
```

- **Algorithm:** Bulk database operations for multiple records
- **Purpose:** Reduce database round trips for batch updates

### 5.3.4 Security Algorithms

#### 5.3.4.1 reCAPTCHA Verification

#### Score-based Assessment:

```
const verifyRecaptcha = async (token) => {
 const response = await fetch('https://www.google.com/recaptcha/api/siteverify', {
 method: 'POST',
 body: new URLSearchParams({
 secret: process.env.RECAPTCHA_SECRET,
 response: token
 })
 });
 const result = await response.json();
 return result.score >= 0.5; // Threshold for human/bot determination
};
```

- **Algorithm:** Machine learning-based risk assessment
- **Purpose:** Distinguish between human users and automated bots
- **Accuracy:** Google's proprietary algorithm with score 0.0-1.0 (1 being the highest chance the user is a human and 0 being the lowest chance, 0.5 the default threshold for human detection )

### 5.3.4 Algorithm Selection Rationale

- **Security Algorithms:** Chosen for industry-standard security (bcrypt, JWT, AES)
- **Performance Algorithms:** Selected for scalability (Redis caching, pagination, indexing)
- **Business Logic Algorithms:** Designed for fairness and transparency (loyalty calculations, tier systems)
- **Data Processing Algorithms:** Optimized for common use cases (filtering, searching, validation)

The algorithms are chosen to balance security, performance, maintainability, and user experience while adhering to industry best practices and compliance requirements.

## 5.4 Security Design

### 5.4.1 Security Features

- **Authentication:** JWT-based authentication with role-based access control (RBAC).
- **Data Encryption:** Sensitive data encrypted at rest and in transit (TLS/SSL).
- **Input Validation:** Comprehensive validation and sanitization of all user inputs.
- **Rate Limiting:** Prevent brute-force attacks using express-rate-limit with Redis store and Redis Lua scripting for dashboard/admin routes.
- **Logging and Monitoring:** SecurityLogs collection for tracking user actions and potential security incidents.
- **Regular Audits:** Periodic security audits and vulnerability assessments.
- **Backup and Recovery:** Planned, not yet implemented (see Roadmap).
- **Compliance:** Adherence to GDPR and other relevant data protection regulations.
- **XSS Protection:** Use of libraries like *xss-clean* and *helmet* to mitigate XSS attacks.
- **HTTP Pollution Protection:** Use of *helmet* and *hpp* to set secure HTTP headers and prevent HTTP parameter pollution.
- **CSRF Protection:** Partial implementation; CSRF tokens are planned for all state-changing operations (see Roadmap).
- **Password Policies:** Strong password requirements enforced via *zxcvbn* and custom validation.
- **Two-Factor Authentication (2FA):** Basic 2FA endpoint implemented; full integration planned (see Roadmap).
- **Session Management:** Secure session handling with appropriate expiration and invalidation.
- **Location-based Security:** Implemented geolocation analysis with risk scoring for fraud detection.
- **reCAPTCHA Integration:** Google reCAPTCHA v3 is integrated into registration and login.
- **IP Hashing:** IP addresses are hashed using SHA-256 before storage in SecurityLogs.

- **NoSQL Injection Prevention:** Use of parameterized queries, Mongoose ODM, and `express-mongo-sanitize`.
- **Security Headers:** `Helmet.js` sets security headers, including CSP.
- **Content Security Policy (CSP):** Strict CSP enforced, no inline scripts except for specific nonces, `eval()` disabled.

### 5.4.2 Security Policies

- **Access Control:** Strict role-based access control (RBAC) to limit user permissions.
- **Data Retention:** Policies for data retention and deletion in compliance with regulations.
- **Incident Response:** Procedures for responding to security incidents and breaches.
- **User Education:** Informing users about security best practices.
- **Regular Updates:** Keeping software and dependencies up to date with security patches.

### 5.4.3 In depth Security Measures

- **Password Hashing:** All passwords are hashed using `bcrypt` with a salt of 10 rounds for both registration and password reset.
- **JWT Authentication:** JSON Web Tokens (JWT) are used for stateless authentication between the client and server. JWT signing secrets are generated using cryptographically secure random values stored in environment variables.
- **Payment Security:** Integration with PayPal's and Google Pay's secure payment gateways, ensuring PCI compliance and encrypted payment data.
- **XSS Protection:** Use of libraries like *xss-clean* and *helmet* to mitigate XSS attacks by sanitizing user inputs and setting secure HTTP headers.
- **CSRF Protection:** Partial; planned for all state-changing operations.
- **Rate Limiting:** Two methods: *express-rate-limit* with Redis store for general routes, and Redis Lua scripting for dashboard/admin routes.

- **IP Hashing:** IP addresses are hashed using SHA-256 before storage in SecurityLogs.
- **reCAPTCHA Integration:** Google reCAPTCHA v3 is integrated into registration and login, using a score-based assessment.
- **NoSQL Injection Prevention:** Parameterized queries, Mongoose ODM, and express-mongo-sanitize are used to prevent NoSQL injection attacks.
- **IP Detection:** Implemented integration with [iplocate.io](https://iplocate.io) for VPN/Proxy/Tor detection and geolocation-based risk analysis.
- **HTTP Parameter Pollution (HPP):** The *hpp* library middleware is used to protect against HTTP Parameter Pollution attacks.
- **CORS Policy:** Strict CORS policy using the *cors* middleware, only allowing requests from the official frontend domain.
- **Security Headers:** Helmet.js sets various HTTP headers for enhanced security.
- **Database Security:** MongoDB connections use authentication with credentials stored in environment variables. Connection pooling and timeouts prevent resource exhaustion. Mongoose ODM provides schema validation and type casting.
- **Error Handling Security:** Sensitive information is never exposed in error messages sent to clients.
- **API Security:** All API endpoints require authentication by default. Public endpoints are explicitly marked and protected with additional rate limiting and bot detection.
- **Environment Security:** All sensitive configuration is stored in environment variables and never committed to version control.

#### 5.4.4 Security Testing and Validation

- **Automated Security Testing:** The project includes security-focused test suites that validate authentication flows, input sanitization, and rate limiting effectiveness.
- **Penetration Testing:** Periodic manual security assessments to identify vulnerabilities not caught by automated tools.

## 6.1 Directory Structure

```
├─ .env
├─ .git/
├─ .gitattributes
├─ .github/
├─ .gitignore
├─ .idea/
├─ .vscode/
├─ code_analytics.json # Code analytics data
├─ config/ # Configuration files
│ ├── DATABASE_CONSTANTS.JS
│ ├── database_queries.js
│ └─ hu.json
├─ data/ # Data files
│ ├── disposable_email_list.json
│ ├── Most_used_passwords.json
│ ├── password_characters.json
│ └─ database_test/ # Test database files
│ ├── Food_Items.json
│ └─ menu_items.json
├─ docs/ # Documentation
│ ├── database.png
│ ├── DatabaseDoc.md
│ ├── DatabaseDoc.pdf
│ ├── Documentation.html
│ ├── Documentation.md
│ ├── Documentation.pdf
│ ├── Paypal_TestDetails.txt
│ ├── RedisLua_README.md
│ ├── snaptraySTACK.png
│ └─ sourcefor_security_checks.txt
├─ node_modules/
└─ package-lock.json
```

```
|─ package.json # Node.js dependencies and scripts
|─ postcss.config.js # PostCSS configuration
|─ public/ # Static files served to client (Frontend)
| |─ favicon.ico
| |─ googlepay.js
| |─ index.html
| |─ password_reset.html
| |─ pay.html
| |─ paypal.js
| |─ register.html
| |─ verify.html
| |─ 404/ # 404 error pages
| |─ 404.html
| └─ 404.jsx
| |─ 429/ # 429 error pages
| |─ 429.html
| └─ 429.jsx
| |─ chat/ # Chat frontend
| |─ dashboard/ # Dashboard pages
| |─ admin/
| |─ parent/
| └─ student/
| |─ home_page/
| |─ information/
| |─ no_perm/
| └─ order/
|─ readme.md # Project documentation
|─ src/ # Server-side source code (backend)
| |─ admin/
| |─ api.js
| |─ auth/
| |─ chapta.js
| |─ dashboard/
| |─ database.js
| └─ examples/
```

```

| ├── logout.js
| ├── LoyaltySystem/
| ├── main.js
| ├── middleware/
| ├── models/
| ├── Orders/
| ├── payments/
| ├── redis-lua.js
| ├── redis.js
| ├── Register.jsx
| ├── script-loader.js
| ├── scripts/
| ├── services/
| └── verificationStore.js
└── tailwind.config.js # Tailwind CSS configuration
 └── tests/ # Test files
 ├── code_analytic.py
 ├── code_analytics.json
 ├── creating_test_users.js
 ├── database_testing.js
 ├── fake_data.py
 ├── menu_items.json
 ├── Paypal_TestConfig.txt
 ├── query_security_logs.js
 ├── register_testing.py
 ├── Jest/
 └── performance_tests/

```

## 6.2 Backend Implementation

### 6.2.1 Technology Stack

The SnapTray backend is built for scalability, security, and maintainability. The main technologies are:



- **Node.js** (runtime)
- **Express.js** (web framework)
- **MongoDB** (NoSQL database)
- **Redis** (in-memory cache, rate limiting, atomic ops)
- **JWT** (authentication)
- **PayPal & Google Pay** (payment gateways)

### 6.2.3 Main Application Structure

- **src/main.js**: Entry point, sets up Express, middleware, routers, and rate limiting.
- **Routers**: Modularized for authentication, dashboard, orders, payments, chat, etc.
- **Database Models**: Defined with Mongoose (see `src/models/` ).
- **Services**: Business logic (loyalty, caching, Redis Lua, payments, etc.).

Example: Express app setup and rate limiting (`src/main.js`)

```
const express = require('express');
const app = express();
const rateLimit = require('express-rate-limit');
const { RedisStore } = require('rate-limit-redis');
const redisClient = require('./redis').redisClient;

const limiter = rateLimit({
 windowMs: 60 * 60 * 1000, // 1 hour
 max: 250,
 store: new RedisStore({
 sendCommand: async (command, ...args) => await redisClient.sendCommand([command, ...args]),
 }),
 handler: (req, res) => {
 res.status(429).sendFile('public/429/429.html');
 },
});
app.use('/api', limiter);
```

## 6.2.4 Authentication & Security

- **Registration:** Validates input, hashes passwords (bcrypt), uses reCAPTCHA, sends verification email, logs security events.
- **Login:** Verifies password, issues JWT, logs IP/location, rate limits by IP.
- **Session:** Stores user info and JWT in session.

Example registration route (src/auth/register.js):

```
router.post('/register', async (req, res) => {
 // ...validation, rate limiting, captcha...
 const hashedPassword = await bcrypt.hash(password, 10);
 const user = new User({ username, password: hashedPassword, email });
 await user.save();
 // ...send verification email...
 res.status(200).json({ message: 'Registration successful!' });
});
```

## 6.2.5 Order & Payment Processing

- **Order API:** Validates cart, checks stock, creates order in DB, integrates with PayPal/Google Pay.
- **Order Completion:** Updates order status, deducts stock, awards loyalty points, logs payment.

Example PayPal order creation (src/api.js):

```
router.post('/orders', async (req, res) => {
 // ...validate user, stock...
 const { jsonResponse, httpStatusCode } = await paypalService.createOrder(cart, currency, amount);
 // ...save order to DB...
 res.status(httpStatusCode).json(jsonResponse);
});
```

## 6.2.6 Caching & Performance

- **Redis:** Used for caching (menu items, sessions), atomic wallet/stock updates, and rate limiting.

- **Cache Middleware:** Wraps API endpoints to serve cached data when available.

Example cache middleware (src/dashboard/services/cache-service.js):

```
function cacheResult(cacheKey, ttl = 300) {
 return async (req, res, next) => {
 if (!isRedisAvailable()) return next();
 const key = typeof cacheKey === 'function' ? cacheKey(req) : cacheKey;
 const cached = await redisClient.get(key);
 if (cached) return res.status(200).json(JSON.parse(cached));
 const originalJson = res.json;
 res.json = function(data) {
 redisClient.setEx(key, ttl, JSON.stringify(data));
 return originalJson.call(this, data);
 };
 next();
 };
}
```

## 6.2.7 Loyalty System

- **Points Calculation:** Points per order, with bonuses for holidays, healthy choices, and user tier.
- **Awarding:** On order completion, points are calculated and added atomically.

Example (src/LoyaltySystem/loyalty-service.js):

```
function ConvertPoints(dollarAmount, tier, healthLevel, date) {
 let total = 0;
 for (let i = 0; i < Math.floor(dollarAmount); i++) {
 total += Math.floor(Math.random() * 6) + 4; // 4-9 points per dollar
 }
 if (isHoliday(date)) total *= 1.5;
 else if (isHolidaySeason(date)) total *= 1.2;
 total *= 1 + (healthLevel * 0.2);
 const tierBonus = DISCOUNT_RATES[tier] || 0;
 total *= 1 + tierBonus;
 return Math.floor(total);
}
```

## 6.2.8 Rate Limiting (Advanced)

- **Express Middleware:** For general API routes.
- **Redis Lua Script:** For dashboard/admin, implements sliding window algorithm for fair, distributed rate limiting.

Example Lua script (used via redis-lua-service):

```

-- KEYS[1]: rate_limit_key
-- ARGV[1]: window_size_seconds, ARGV[2]: max_requests, ARGV[3]: current_timestamp
local key = KEYS[1]
local window = tonumber(ARGV[1])
local max_requests = tonumber(ARGV[2])
local now = tonumber(ARGV[3])
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local current_count = redis.call('ZCARD', key)
if current_count >= max_requests then
 return {0, current_count}
end
redis.call('ZADD', key, now, now)
redis.call('EXPIRE', key, window)
return {1, current_count + 1}

```

## 6.2.9 Extensibility & Maintainability

- Modular folder structure: `src/auth/` , `src/dashboard/` , `src/services/` , etc.
- All critical operations are logged.
- Admin routes protected by middleware and advanced rate limiting.

**[Insert additional diagrams here as needed, e.g., sequence diagrams for order/payment flow, caching, or loyalty point calculation.]**

# Router Modules and API Endpoints

---

-- Set expiration on the key (cleanup)

```
redis.call('EXPIRE', key, window)
```

```
return {1, current_count + 1} -- 1 = allowed, new count
```

- The Lua script implements a sliding window rate limiting algorithm using Redis sorted sets to track request timestamps.  
(The sliding window algorithm provides a more accurate rate limiting mechanism compared to fixed window algorithms by allowing requests to be counted over
- The script uses Redis commands like `ZREMRANGEBYSCORE` to remove old entries, `ZCARD` to count current requests, and `ZADD` to add new request timestamps
- It returns a status indicating whether the request is allowed or blocked, along with the current request count.
- The script is executed atomically, ensuring consistent rate limiting even under high concurrency.
- There is a line called `@diagnostic disable: undefined-global` which is used to disable warnings from code analysis tools about undefined global variable

**\*\*Router Modules and API Endpoints:\*\***

## Router Modules and API Endpoints {#router-modules-and-api-endpoints }

### Main Application Routes {#main-application-routes }

Method	Endpoint	Description
GET	/login	Login page
GET	/register	Registration page
GET	/password-reset/:token	Password reset page
GET	/pay	Payment page
GET	/chat	Chat frontend (E2EE chat UI)

### Authentication Routes {#authentication-routes }

Method	Endpoint	Description
POST	/register	User registration
POST	/login	User login
POST	/logout	User logout
GET	/logout	Logout confirmation
POST	/2fa	Two-factor authentication

### Email Verification Routes {#email-verification-routes }

Method	Endpoint	Description



	POST		`/email-verification/verify-code`		Verify email code	
	GET		`/email-verification/verify/:token`		Verify email with token	

### ### Password Reset Routes {#password-reset-routes }

	Method		Endpoint		Description	
	-----		-----		-----	
	POST		`/password-reset/`		Request password reset	
	GET		`/password-reset/:token`		Password reset form	
	POST		`/password-reset/:token`		Submit new password	
	POST		`/forgot-password/`		Forgot password request	

### ### Dashboard Routes {#dashboard-routes }

	Method		Endpoint		Description	
	-----		-----		-----	
	GET		`/dashboard/`		Main dashboard	
	GET		`/dashboard/admin`		Admin dashboard page	
	GET		`/dashboard/student`		Student dashboard page	

### ### Admin Dashboard API Routes {#admin-dashboard-api-routes }

	Method		Endpoint		Description	
	-----		-----		-----	
	GET		`/dashboard/admin/usercount`		Get user count	
	GET		`/dashboard/admin/userlist`		Get list of users	
	GET		`/dashboard/admin/stats`		Get admin statistics	
	GET		`/dashboard/admin/signup-stats`		Get signup statistics	
	GET		`/dashboard/admin/orders`		Get orders data	
	GET		`/dashboard/admin/soldout`		Get sold out items	
	GET		`/dashboard/admin/itemcount`		Get item count	
	GET		`/dashboard/admin/menulist`		Get menu items list	
	GET		`/dashboard/admin/stockalerts`		Get stock alerts	
	GET		`/dashboard/admin/paymentstats`		Get payment statistics	
	GET		`/dashboard/admin/welcome-message`		Get welcome message	

	GET		`/dashboard/admin/health`		System health check	
	GET		`/dashboard/admin/menuitem_export`		Export menu items	
	GET		`/dashboard/admin/delete_menuitem/:id`		Delete menu item	
	POST		`/dashboard/admin/create_menuitem`		Create new menu item	
	PUT		`/dashboard/admin/menuitem/:id`		Update menu item	

### Student Dashboard Routes {#student-dashboard-routes }

	Method		Endpoint		Description	
	-----		-----		-----	
	GET		`/dashboard/student/freeze_account`		Freeze student account	
	POST		`/dashboard/student/parent/link`		Link parent account	

### Order Management Routes {#order-management-routes }

	Method		Endpoint		Description	
	-----		-----		-----	
	GET		`/Order/`		Order page	
	GET		`/Order/menu_items`		Get menu items for ordering	
	GET		`/Order/:orderId`		Get specific order details	
	POST		`/Order/Order`		Create new order	
	PUT		`/Order/:orderId/status`		Update order status	
	POST		`/Order/:orderId/capture`		Capture order payment	

### Admin Management Routes {#admin-management-routes }

	Method		Endpoint		Description	
	-----		-----		-----	
	GET		`/admin/changeuser`		Change user permissions	

---

## API Routes {#api-routes }

### General API Routes {#general-api-routes }

Method	Endpoint	Description
GET	`/api/test`	API test endpoint
GET	`/api/current_user`	Get current logged-in user
GET	`/api/menu-items`	Get available menu items

### ### Order API Routes {#order-api-routes }

Method	Endpoint	Description
POST	`/api/orders`	Create PayPal order
POST	`/api/orders/:orderId/capture`	Capture PayPal payment

### ### Google Pay API Routes {#google-pay-api-routes }

Method	Endpoint	Description
POST	`/api/orders/googlepay`	Create Google Pay order
POST	`/api/orders/googlepay/complete`	Complete Google Pay transaction

### ### Payment Integration Routes {#payment-integration-routes }

Method	Endpoint	Description
POST	`/api/payments/paypal`	PayPal payment processing
POST	`/api/payments/googlepay`	Google Pay payment processing

#

### ### Chat API & WebSocket Routes {#chat-api--websocket-routes }

Method	Endpoint	Description
GET	`/chat`	Chat frontend (serves chat UI)
WS	`/chat`	WebSocket endpoint for real-time chat
POST	`/chat/setup-e2ee`	Setup E2EE public key for user

GET	`/chat/public-key/:userId`	Get E2EE public key for a user
POST	`/chat/send-message`	Send encrypted message (E2EE)
GET	`/chat/messages/:otherUserId`	Get messages for a conversation
GET	`/chat/message/:messageId`	Fetch a single message
POST	`/chat/message/:messageId/replace`	Mark a message as replaced (auto-resend)
PUT	`/chat/messages/read/:otherUserId`	Mark messages as read
GET	`/chat/conversations`	Get conversation list
GET	`/chat/search-users`	Search users for chat
GET	`/chat/e2ee-status`	Get E2EE status for current user
POST	`/chat/reset-e2ee`	Reset E2EE keys for user
POST	`/chat/backup-keys`	Backup encrypted private key to server
GET	`/chat/has-key-backup`	Check if user has a key backup
GET	`/chat/restore-keys`	Restore encrypted private key from server
	<span style="color:#d32f2f;font-weight:bold">POST</span>   <span style="color:#d32f2f;font-weight:bold">`/chat/admin/clear-all-e2ee`</span>	
POST	`/chat/request-sender-recovery`	Mark messages as needing sender-key recovery
GET	`/chat/pending-recovery`	Get messages needing sender-key recovery

#### **\*\*WebSocket events:\*\***

- `newMessage` (real-time message delivery)
- `messageReplaced` (auto-resend notification)
- `processPendingRecovery` (sender-key recovery)

<span style="color:#d32f2f;font-weight:bold">**\*\*Admin-only routes are shown in red and are not accessible to normal users.\*\***</span>

#### **\*\*Notes:\*\***

- All chat API endpoints require authentication.
- E2EE = End-to-End Encryption (all messages are encrypted client-side).
- WebSocket endpoint is used for real-time chat updates and notifications.

#### **# 6.2.3 Database Integration and Models {#623-database-integration-and-models }**

The backend uses MongoDB as the primary database for storing user data, orders, menu items, and security logs. Mongoose is used as the ODM (Object Data Mod

#### **## User Model {#user-model }**

- **\*\*username\*\***: String, required, unique
- **\*\*password\*\***: String, required, hashed with bcrypt (10 rounds)

- **\*\*email\*\***: String, required, unique, validated
- **\*\*isVerified\*\***: Boolean, default false
- **\*\*usertype\*\***: String, enum: ['admin', 'student', 'parent', 'teacher', 'frozen', 'editor'], default: 'student'
- **\*\*createdAt\*\***: Date, default now
- **\*\*balance\*\***: Number, default 0
- **\*\*isBanned\*\***: Boolean, default false
- **\*\*banReason\*\***: String, default ''
- **\*\*userPersonalInfo\*\***: Subdocument for personal info (firstName, lastName, etc.)
- **\*\*identity\*\***: E2EE identity fields
- **\*\*devices\*\***: Registered device info
- **\*\*recoveryBlob\*\***: For encrypted recovery data
- **\*\*encryption\*\***: Legacy E2EE fields

#### ## Loyalty System {#loyalty-system }

- Points are awarded per order using a random value (4-9 points per dollar spent), with bonuses for holidays and healthy choices.
- Tiers: NONE, BRONZE, SILVER, GOLD, PLATINUM (see config/DATABASE\_CONSTANTS.JS for rates and names)
- Tier bonuses and discounts are applied automatically.
- See `src/LoyaltySystem/loyalty-service.js` for calculation details.

#### ## Admin Dashboard Features {#admin-dashboard-features }

- Advanced rate limiting for all admin routes (30 requests/minute, Redis Lua-based)
- User management: list, count, ban, change roles
- Menu and order management: view, update, export
- Statistics and analytics endpoints
- All endpoints are protected by admin permission middleware

#### ## Caching Strategy {#caching-strategy }

The backend employs Redis as an in-memory data store to cache frequently accessed data, such as menu items and user sessions. This reduces database load and

- The project uses the following redis keys as seen in: `src/cache/KeyRegistry.js`

```
```javascript
```

```
const keyRegistry = {
```

```
  /*
```

These are the keys that are in a MongoDB database collection (tables)

*/

```
users: (userId) => [  
  `user:username:${userId}`,  
  `student:userinfo:${userId}`,  
  'admin:usercount',  
  'admin:userlist',  
  'admin:activeusers',  
  'admin:signup-stats',  
  'admin:stats',  
  'editor:usercount',  
  'editor:activeusers',  
],
```

```
menuitems: (itemName) => [  
  `menu_item:${itemName}`,  
  'admin:menulist',  
  'admin:itemcount',  
  'admin:stockalerts',  
  'admin:soldout',  
  'admin:menuitem_export',  
  'editor:menulist',  
  'editor:itemcount',  
],
```

```
dailymenus: () => [  
  'daily_menu:available',  
  'admin:menulist',  
  'editor:menulist',  
],
```

```
orders: (orderId) => [  
  `order:${orderId}`,  
  'admin:orders',
```

```
'editor:orders',  
'admin:most_bought_items_alltime',  
'admin:most_bought_items_lastweek',  
'admin:average_order_value',  
],
```

```
orderitems: (userId) => [  
  `student:order_history:${userId}`,  
  `parent:orders:${userId}`,  
],
```

```
payments: (userId) => [  
  `student:transactions:${userId}`,  
  `parent:transactions:${userId}`,  
  'admin:paymentstats',  
  'admin:revenue_lastmonth',  
  'admin:total_revenue',  
  'admin:stats',  
],
```

```
userloyalties: (userId) => [  
  `student:loyalty:${userId}`,  
  `loyalty:rewards:${userId}`,  
  `wallet:user:${userId}`,  
  `student:wallet_balance:${userId}`,  
  `student:wallet:${userId}`,  
  'admin:totalpoints',  
  'editor:totalpoints',  
  `parent:stats:${userId}`,  
],
```

```
rewards: () => [  
  'admin:rewards_list',  
  'admin:reward_stats',  
  'editor:rewards_list',
```

```
    'editor:reward_stats',  
  ],
```

```
redemptions: (userId) => [  
  `student:loyalty:${userId}`,  
  `loyalty:rewards:${userId}`,  
  'admin:reward_stats',  
  'editor:reward_stats',  
],
```

```
parentstudents: (userId) => [  
  `parent:studentlist:${userId}`,  
  `parent:welcome:${userId}`,  
  `parent:link-requests:${userId}`,  
  `parent:stats:${userId}`,  
],
```

```
prekeys: (userId) => [  
  `e2ee:pubkey:${userId}`,  
],
```

```
// ————— Does not fit into any collection, but still requires caching ————— //
```

```
verification: (email) => [  
  `verification:${email}`,  
],
```

```
loginAttempts: (ip) => [  
  `login_attempts:${ip}`,  
],
```

```
regAttempts: (ip) => [  
  `reg_attempts:${ip}`,  
],
```



```
rateLimit: (role, identifier) => [  
  `ratelimit:${role}:${identifier}`,  
],  
  
chatRateLimit: (userId) => [  
  `rate_limit:user:${userId}`,  
],  
  
system: () => [  
  'system:health',  
],  
  
};
```

13. Planned Features and Roadmap

- Multi-language support (Hungarian/English)
- In-app feedback and bug reporting
- Push/pop notifications (WebSocket/SSE, SMS/email alerts)
- Enhanced menu photos, nutritional info, allergen warnings
- QR code integration for menu/table ordering
- Super admin approval for critical changes
- Admin impersonation and activity history
- Admin rate limit dashboard (change limits from UI)
- Password breach check (integration with breach databases)
- Biometric authentication (fingerprint/face)
- Advanced fraud detection (ML-based, IP geolocation anomaly)
- Analytics and reporting (email summaries, user data export, API usage, health routes)
- Advanced analytics dashboard (nutritional tracking, spending analysis, predictive analytics)
- Parent spending analytics
- Smart ordering features and menu recommendation engine
- Enhanced dietary management
- Achievement system and badges

- Social/community features

7. Testing and Validation

8. User Manual

9. Deployment and Maintenance

10. Conclusion and Future Work

11. References

12. Appendices
